09

Third-party integrations

Calling services you don’t control, and handling their failures

Payments, email, maps, AI models: most apps rent these as services, reached by calls across the internet to systems with their own outages, their own limits, and their own release schedules. Generated integration code reliably handles the case where the call works; the failures live in the cases where it half-works, takes too long, or gets sent twice. This chapter covers the small set of concepts those cases turn on.

Assumes chapter 4’s request path; idempotency is introduced in chapter 3.

PART

The model

What an integration is

An integration is your server calling theirs. You control the request you send and nothing after it — their latency, their availability, whether their answer arrives at all. Their bad day reaches your users as your symptom — a blank screen, a spinner that never resolves — unless your side has decided in advance what to do when the answer doesn't come.

Timeouts, retries, backoff

Three settings govern the waiting, and each either exists or doesn't — which makes them enumerable.

A **timeout** is how long you wait before giving up. Without one, the wait is unbounded and the user waits with it.

A **retry** is trying again, sensible only for failures that might pass — a network blip, a momentary overload — and not for a rejection that will repeat.

Backoff is waiting longer between attempts. Immediate retries from every caller are how a struggling service gets pushed over; spacing them is the polite and the effective version.

Partial failure

A call can half-work, and the halves matter. Your write succeeded and their notification failed: the data is right and the user was never told. Their charge succeeded and your record of it failed: money moved and your database doesn't know.

The case worth special attention is the timeout, because a timeout is not a failure — it's an unknown. The request may have landed and been processed; only the answer is missing. Code that treats "no answer" as "didn't happen" will retry things that happened, which is the next section's problem.

DIAGRAM

two boxes, yours and theirs, with a request arrow across and a response arrow back; the response arrow cut partway, and a question mark on each side labelled “did it happen?”

Repeating a call that creates

Retries, timeouts, double-clicks, and webhook redeliveries all mean the same creating call can be sent more than once. Run naively, a call sent twice does its work twice: two charges, two records, two emails. The guard is the **idempotency key** from chapter 3, applied across the boundary: you supply an identifier with the call, and the receiving service treats any repeat carrying the same identifier as the same operation. Payment providers support this in their APIs precisely because the double-charge is that common; using it is a decision your integration code either made or didn’t, which makes it askable.

Rate limits

Services cap how often you may call them, and reaching the cap is a normal operating state rather than an error: the correct responses are backing off, queueing the work (chapter 4), or spreading it out. Code that treats a rate-limit response as a crash converts a routine throttle into an outage. The mirror — capping how often others may call *your* endpoints, which is what keeps a paid call from becoming a bill — is chapter 4’s unmetered-endpoint entry.

Change on their schedule

An external service changes on its own timeline: versions, deprecations, new required fields. A dependency therefore costs more than the integration work — it adds their calendar to yours, and eventually a migration. This is the real weight behind “should we add this service,” and it’s a chapter 12 tier question as much as a technical one.

Their calls to you

Webhooks (chapter 4) are the arrangement reversed, and two properties matter on the receiving end. The URL is public, so the handler verifies the sender — providers sign their deliveries for exactly this. And deliveries repeat by design — providers resend when unsure you received one — so processing a webhook must be idempotent too — the same two concepts, arriving from the other direction.

Their code, in your process

A library is the other way of using someone else's work: their code runs inside your process, with your permissions, reading whatever your code can read. Adding one is cheaper than adding a service and carries the same calendar — updates arrive on their schedule, abandonment happens on theirs — plus one property a service lacks: whatever a library does when it runs, it does as you. Generated code adds libraries freely, since importing one is shorter than writing the function, and the addition usually goes unmentioned in the summary. Three questions cover it at recognition depth: what does this do for us, could we do without it, and is anyone maintaining it.

What you send them

Every call carries data out of your system into theirs, and for some calls that data is your users': the text they typed, sent to a language model; their email, sent to a delivery service; their address, sent to a map. From that point it lives under the other service's retention and the other service's rules, and your users, in most places, have a right to know that and a right to have it deleted. The decision worth making on purpose is which services receive personal data and whether each needs it — a model asked to summarize a document doesn't need the author's email attached.

PART

What goes wrong

1 No handling of partial failure

the integration has one plan, and it's success.

ASK "For each external call, state the timeout, the retry policy, and what the user sees when it fails. Say none or undecided explicitly."

CHECK point the call at an address that doesn't answer, and use the feature.

TELL *an external call with no timeout, no retry policy, and no failure branch.*

2 Retry without idempotency

trying again can mean doing it twice.

ASK "For each call that can be sent more than once — retries, double-clicks, webhook redeliveries — say whether running it twice produces one result or two."

CHECK run it twice with the same input and count what exists afterwards.

TELL *a retry wrapping a call that creates something, with no idempotency key.*

3 The unexamined dependency

a library added to do one thing, with nobody having asked what else it does or who maintains it.

ASK "List every dependency this change added, with what it's used for, whether the project could do the same without it, and when it was last published. Flag anything used for one function and anything not published in the last year."

CHECK the dependency list before and after the change, read against the summary.

TELL *new entries in the dependency list that the summary never mentioned; a package imported for a single function.*

4 User data leaving without a decision

personal data sent to a service that didn't need it.

ASK "For each external call, list which fields about a person it sends, and which of them the service needs to do its job. Flag every call that sends personal data it doesn't use."

CHECK the request body of one call, read in the network tab or the server log, against what the service does with it.

TELL *whole records passed to an external call that uses one field of them.*

THE DIRECT TEST

Point one integration at an address that doesn't answer and use the feature as a user would. What the screen does while nothing comes back, and what state the data is in afterwards, is your app's actual partial-failure behaviour.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · The integration inventory

AUDIT

List every external service my app calls, and every call to it. For each call: the timeout, the retry policy, the backoff, whether an idempotency key is used, which fields about a person it sends, and what the user sees when the call fails. Full table; write none where the answer is none.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a table with honest nones — a young integration usually has a row that reads none, none, none, and that row is the worklist. Follow up on any creating call without a key: "show me what two of these would look like in the data."

D2 · Walk the unknown outcome

WALK THE FAILURE PATH

Walk me through this case: my app calls [the payment provider] to charge a customer, the charge succeeds on their side, and the response never reaches me — a timeout. Step by step: what each system believes at that moment, what my code does next, what the user sees, and what eventually reconciles the two sides, if anything does.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response treats the timeout as an unknown rather than a failure, and ends at a named reconciliation — a webhook, a scheduled comparison, or an honest nothing. "Nothing reconciles it" is a real and common finding.

D3 · The dead-address drill

BUILD A TOY

Help me run the direct test on [one integration]: point it at an address that doesn't answer in a test copy, use the feature, and watch what happens — how long the wait is, what the screen shows, what lands in the data. Then put it back and run the creating call twice with the same input, and count.

WHAT A GOOD RESPONSE LOOKS LIKE

Both halves take about half an hour. The wait's length is your effective timeout — unbounded is a finding — and the count after the double call is your idempotency answer.

D4 · The failure quiz

PREDICTION QUIZ

Quiz me on my integrations. One at a time, describe a realistic misbehaviour — their service is down for an hour, a response arrives after forty-five seconds, the same webhook is delivered three times, we hit their rate limit at peak — and ask me what my app does. Wait for my answer, then tell me what it actually does, per the code.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit before each correction. The webhook-redelivery case is the one most apps fail quietly — it's the retry-without-idempotency entry arriving from outside.

WHERE THIS CONNECTS

Chapter 3 · Source of truth introduces idempotency; this chapter applies it across the boundary. **Chapter 4 · The request lifecycle** owns webhooks and the queues that rate-limited work waits in. **Chapter 8 · Injection** covers what happens to the text you send a language model. **Chapter 11 · The loop** puts the dependency list into the check that runs on every diff. **Chapter 12 · Verification** is how claims like "it retries safely" get settled.